

EduGenie: Google Gemini Powered Learning Assistant

Project Documentation

Project Detail	Information
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Team ID	6ab39579211fa4fd5d28901b
Team Lead	THAMIJUDEEN M
Team Members	THAMIJUDEEN M; SUNDAR S; SENTHIL KUMARAN S; SUDHARSANA SARANGI L, SUNILKUMAR
Documentation Date	30 September 2026

1. Project Description

EduGenie is a GoogleGemini Powered Learning Assistant designed to support college students during technical learning. The application brings multiple learning activities into one web interface so that a student can ask questions, simplify difficult concepts, practice with quizzes, summarize study material, and receive a personalized learning path.

The project uses a modular AI architecture. Gemini 3.5 Flash-Lite is used for Question Answering, Quiz Generation, Summarization, and Personalized Learning Path generation. LaMini-Flan-T5-783M is used locally for Concept Explanation. This allocation separates the general-purpose generative tasks from the local concept-explanation module.

EduGenie is implemented as a FastAPI application with an HTML/CSS frontend. The application is intended to run locally during the project demonstration. No persistent database is used in the current MVP.

Core capabilities

- Question Answering — provides AI-generated answers to student questions.
- Concept Explanation — provides simplified technical explanations with examples.
- Quiz Generation — generates topic-based questions for practice and self-assessment.
- Summarization — converts submitted learning content into a concise summary.
- Personalized Learning Path — creates a structured learning plan based on subject, level, goal, available time, and duration.

Project objective

The objective is to provide a single, practical learning assistant that reduces the need to switch between separate learning resources for explanation, practice, summarization, and planning.

2. Usage Scenarios

Scenario 1 — Technical Question

A student has a question about a programming or computer-science topic. The student enters the question in EduGenie. The FastAPI backend validates the request and sends it to Gemini 3.5 Flash-Lite. The generated answer is returned to the web interface.

Scenario 2 — Difficult Concept

A student finds a technical concept difficult to understand. The student enters the concept and optional context. EduGenie uses the local LaMini-Flan-T5-783M model to generate a simplified explanation with a relevant example.

Scenario 3 — Practice Quiz

A student wants to practice a topic before an examination. The student selects a topic, number of questions, and difficulty. EduGenie generates structured quiz questions through Gemini 3.5 Flash-Lite.

Scenario 4 — Study Material Summary

A student has lengthy notes on educational content. The content is submitted to EduGenie with a requested summary length. Gemini produces a concise summary containing the important points.

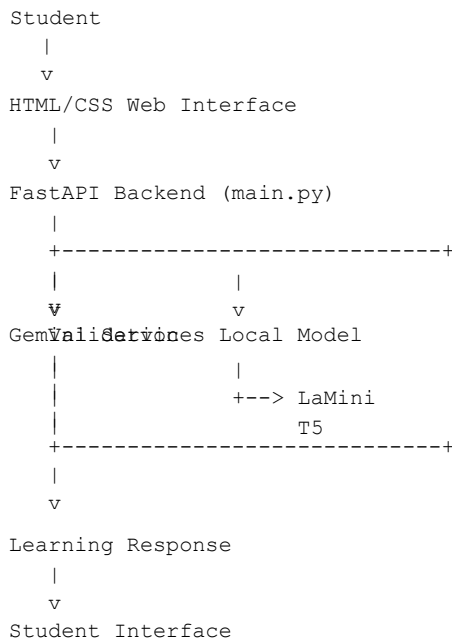
Scenario 5 — Personalized Learning Path

A student specifies the subject, current level, learning goal, available time, and duration. EduGenie generates a structured learning path that organizes study activities across the requested period.

3. Technical Architecture

EduGenie follows a modular web application architecture in which the frontend communicates with a FastAPI backend. The backend routes each learning request to the appropriate service module and AI model.

Text-based architecture flow



Model allocation

Feature	Model	Reason / Role
Question Answering	Gemini 3.5 Flash-Lite	General educational question answering
Quiz Generation	Gemini 3.5 Flash-Lite	Structured question generation
Summarization	Gemini 3.5 Flash-Lite	Concise educational summaries
Personalized Learning Path	Gemini 3.5 Flash-Lite	Structured learning-plan generation
Concept Explanation	MBZUAI/LaMini-Flan-T5-783M	Local simplified concept explanation

Database note

No persistent database is used in the current EduGenie MVP. Requests are processed and results are returned through the application without storing user learning history in a database.

4. Pre-requisites

The project requires a Python development environment, the FastAPI web framework, the Google Gemini SDK, Jinja2 templates, Uvicorn, and the local transformer model dependencies used by the concept-explanation module.

Requirement	Purpose
Python 3.10+	Application development and execution
FastAPI	Backend API framework
Uvicorn	ASGI development server
Jinja2	HTML template rendering
google-genai	Gemini API integration
python-dotenv	Secure environment-variable loading
transformers	Local LaMini model integration
PyTorch CPU	Local model execution
Git / GitHub	Version control and repository management
Modern web browser	Running and testing the frontend

Environment configuration

The Gemini API key is stored in a local **.env** file. The file is excluded from Git tracking through **.gitignore**. A safe **.env.example** file is included for setup guidance.

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

The actual API key must never be included in the repository or documentation.

5. Project Workflow

The project follows the SkillWallet milestone-oriented workflow used for development and documentation.

Milestone 1: Model Selection and Architecture

- Configure the Gemini API environment securely.
- Evaluate available Gemini model options and select a working model for the project.
- Select Gemini 3.5 Flash-Lite for the Gemini-powered features.
- Select MBZUAI/LaMini-Flan-T5-783M for local concept explanation.
- Define the modular architecture and model-to-feature allocation.

Milestone 2: Core Functionalities Development

- Implement Question Answering.
- Implement Concept Explanation.
- Implement Quiz Generation.
- Implement Summarization.
- Implement Personalized Learning Path generation.

Milestone 3: Frontend Development

- Build the web interface using HTML and CSS.
- Create input areas for each learning feature.
- Connect frontend actions to FastAPI endpoints.
- Display generated outputs clearly to the student.

Milestone 4: Testing and Deployment

- Run the application locally using Uvicorn.
- Test each functional feature.
- Verify API integration and frontend responses.
- Document the final project and repository.

6. Milestone 1 — Model Selection and Architecture

The first milestone focused on selecting AI models that fit the educational requirements of EduGenie while keeping the implementation practical for local development.

Activity 1.1 — Gemini API configuration

A Gemini API key was created through Google AI Studio. The key is loaded through the environment configuration rather than being hard-coded into source files. The project uses the official Google Gen AI Python SDK.

Activity 1.2 — Model selection

The selected Gemini model for the submitted implementation is **gemini-3.5-flash-lite**. It is used for Question Answering, Quiz Generation, Summarization, and Personalized Learning Path generation.

Activity 1.3 — Local concept-explanation model

The concept-explanation feature uses **MBZUAI/LaMini-Flan-T5-783M**. The model is downloaded and executed locally through the Transformers and PyTorch stack.

Activity 1.4 — Central model configuration

```
GEMINI_MODEL = "gemini-3.5-flash-lite"
LOCAL_EXPLANATION_MODEL = "MBZUAI/LaMini-Flan-T5-783M"
```

The model configuration is centralized in **model_config.py** so feature modules use consistent model assignments.

7. Milestone 2 — Core Functionalities Development

The backend is divided into independent feature modules. Each module validates its input and returns a feature-specific result to the FastAPI route.

Question Answering — `qna.py`

The Q&A module validates the question, loads the Gemini API configuration, sends a focused educational prompt to Gemini 3.5 Flash-Lite, and returns the generated answer.

Concept Explanation — `explanation_module.py`

The concept-explanation module validates the concept and optional context, tokenizes the request, runs the LaMini-Flan-T5-783M model locally, and decodes the generated explanation.

Quiz Generation — `quiz_module.py`

The quiz module asks Gemini to return a structured quiz and validates the returned structure before sending the questions to the API layer.

Summarization — `summary_module.py`

The summarization module validates the content and requested summary length, then uses Gemini to produce a concise educational summary.

Personalized Learning Path — `learning_path.py`

The learning-path module accepts subject, current level, learning goal, available time, and duration. Gemini generates a structured weekly plan that is validated before being returned.

8. Backend Development with FastAPI

The FastAPI application is defined in **main.py**. It creates the application, serves the HTML interface, mounts static files, validates request bodies using Pydantic models, and connects API routes to the service modules.

API endpoints

Method	Endpoint	Purpose
GET	/	Serve the EduGenie web interface
GET	/health	Return application health status
POST	/qa	Question Answering
POST	/explain	Concept Explanation
POST	/quiz	Quiz Generation
POST	/summarize	Summarization
POST	/learn/recommendations	Personalized Learning Path

Request validation

Pydantic request models enforce input boundaries for question length, concept length, context length, quiz count, difficulty, summary content, and learning-path parameters. Invalid client input returns a 400-level validation response through the API layer.

Error handling

Service-level `ValueError` conditions are returned as HTTP 400 responses, while unexpected service exceptions are returned as HTTP 500 responses. This keeps API failures visible to the frontend without exposing the secret API key.

9. Milestone 3 — Frontend Development

The EduGenie frontend is implemented using an HTML template and a dedicated CSS stylesheet. The interface is designed as a single learning-assistant page with sections for the five core capabilities.

Frontend files

```
templates/  
  ... index.html  
  
static/  
  ... style.css
```

Interface sections

Section	Purpose
Navigation / Hero	Introduce EduGenie and its learning-assistant purpose
Question Answering	Accept a student question and display the generated answer
Concept Explanation	Accept a concept and optional context and display an explanation
Quiz	Collect topic, question count, and difficulty and display generated questions
Summarization	Accept learning content and display the requested summary
Learning Path	Collect planning inputs and display a personalized learning plan
Footer / CTA	Provide closing project information

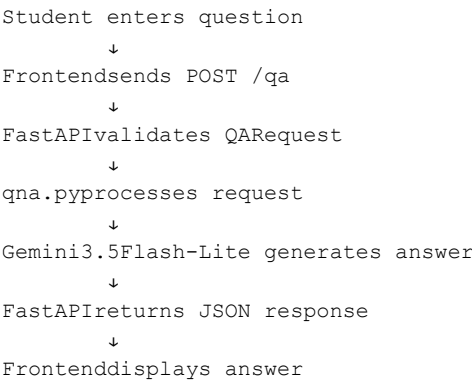
Frontend design

The interface uses a clean light visual style with clear cards, responsive spacing, readable typography, form controls, result sections, and hover/loading states. The goal is to make the five AI functions understandable without requiring technical knowledge from the student.

10. Live Integration

The frontend and backend are integrated through HTTP requests to the FastAPI endpoints. A typical interaction starts when the student submits a form, continues through request validation and feature processing, and ends when the generated result is rendered in the browser.

Q&A; integration flow



Other features follow the same integration pattern while routing to their corresponding module and model.

Model routing

Endpoint	Module	Model
/qa	qna.py	Gemini 3.5 Flash-Lite
/explain	explanation_module.py	LaMini-Flan-T5-783M
/quiz	quiz_module.py	Gemini 3.5 Flash-Lite
/summarize	summary_module.py	Gemini 3.5 Flash-Lite
/learn/recommendations	learning_path.py	Gemini 3.5 Flash-Lite

11. Milestone 4 — Local Execution

EduGenie is designed to run locally for development and demonstration. The project does not require a cloud deployment for the current submission.

Environment setup

```
python --version
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Environment file

Create `.env` in the project root using `.env.example` as a reference and add the Gemini API key.

Start the application

```
python-muvicornmain:app --reload
```

Open the application

```
http://127.0.0.1:8000/
```

Static file route

The stylesheet is served through FastAPI's static-file mount at `/static/style.css`.

12. Functional Testing and Verification

Functional testing focused on verifying that each of the five core EduGenie features accepts valid input and produces the expected type of result through the integrated application.

Feature	Test Input	Expected Result	Observed Result	Status
Question Answering	Technical question	Clear AI-generated answer	Answer generated successfully	Pass
Concept Explanation	Technical concept	Simplified explanation with example	Explanation generated successfully	Pass
Quiz Generation	Topic + count + difficulty	Structured quiz questions	Quiz generated successfully	Pass
Summarization	Educational content + length	Concise summary	Summary generated successfully	Pass
Personalized Learning Path	Subject + level + goal + time + duration	Structured learning plan	Learning path generated successfully	Pass

Integration verification

The application was run locally and the five feature flows were manually exercised through the web interface. The API responses were returned to the frontend and rendered in the corresponding result sections.

Testing result

All five core functional features included in the current MVP were verified as working during the documented functional-testing stage.

13. Project Structure and Repository

The project repository contains the application source, documentation, configuration examples, and dependency file.

```
EduGenie/
|
|-- main.py
|-- model_config.py
|-- qna.py
|-- explanation_module.py
|-- quiz_module.py
|-- summary_module.py
|-- learning_path.py
|
|-- templates/
|   |-- index.html
|
|-- static/
|   |-- style.css
|
|-- docs/
|   |-- 01-prerequisites/
|   |-- 02-project-workflow/
|   |-- 03-model-selection/
|   |-- 04-module-implementation/
|   |-- 05-backend-api/
|   |-- 06-frontend-interface/
|   |-- 07-live-integration/
|   |-- 08-run-locally/
|   |-- 09-functional-testing/
|   |-- 10-conclusion/
|
|-- requirements.txt
|-- .env.example
|-- .gitignore
|-- README.md
```

GitHub repository

<https://github.com/thamijudeen1706/EduGenie-Gemini-Learning-Assistant>

Repository documentation

The repository includes a professional README and milestone-wise documentation covering prerequisites, workflow, model selection, implementation, backend API, frontend, live integration, local execution, functional testing, and conclusion.

14. Security and Configuration

The current MVP uses environment-based configuration for the Gemini API key. The secret is not hard-coded into source files and the .env file is excluded from Git tracking.

Configuration files

File	Purpose
.env	Local secret configuration; not committed
.env.example	Safe configuration template without the real secret
.gitignore	Excludes .env, .venv, and Python cache files
model_config.py	Central model allocation and project model metadata
requirements.txt	Python dependency list

No user authentication

The current MVP does not implement user registration or login. This is intentional for the present learning-assistant scope and does not represent a database-backed user-management system.

No persistent storage

EduGenie does not currently store user questions, generated answers, quiz history, summaries, or learning paths in a persistent database.

15. Code Organization and Maintainability

The implementation separates responsibilities into feature modules instead of placing all AI logic inside the main FastAPI file.

Component	Responsibility
main.py	FastAPI application, request models, routes, templates, static files
model_config.py	Central model names, assignments, purposes, and lookup functions
qna.py	Question Answering service
explanation_module.py	Local concept-explanation service
quiz_module.py	Structured quiz generation and validation
summary_module.py	Educational summarization
learning_path.py	Personalized learning-path generation
templates/index.html	Web interface
static/style.css	Frontend styling

Reusability

Each service exposes a focused Python function that can be called from the corresponding FastAPI route. This separation makes it easier to test or modify one learning feature without rewriting the entire application.

Readability

The source uses descriptive module and function names, explicit validation, centralized model configuration, and a consistent API response structure.

16. API Reference

Endpoint	Request Highlights	Response Highlights
/qa	question	success, feature, question, answer
/explain	concept, context	success, feature, concept, explanation
/quiz	topic, number_of_questions, difficulty	success, feature, topic, difficulty, questions
/summarize	content, summary_length	success, feature, summary_length, summary
/learn/recommendations	subject, current_level, learning_goal, available_time, duration_weeks	success, feature, learning_path

Health endpoint

GET /health

Response:

```
{
  "status": "healthy",
  "application": "EduGenie"
}
```

API design

The endpoints use JSON request and response bodies. The backend validates inputs before calling the corresponding service. Successful responses include a success flag and feature-specific result data.

17. Demonstration Flow

The recommended demonstration follows a simple three-part sequence.

Part 1 — Project and Code

- Show the EduGenie repository/project folder.
- Open README.md and briefly explain the project.
- Show model_config.py and the Gemini / LaMini model allocation.
- Show main.py and the FastAPI routes.
- Show the terminal running Uvicorn.

Part 2 — Frontend

- Open `http://127.0.0.1:8000/`.
- Briefly show the five learning features in the interface.
- Explain that the interface communicates with the FastAPI backend.

Part 3 — Feature Outputs

- Demonstrate Question Answering.
- Demonstrate Concept Explanation.
- Demonstrate Quiz Generation.
- Demonstrate Summarization.
- Demonstrate Personalized Learning Path.

Security reminder

The Gemini API key must never be displayed during the demonstration. The .env file should remain hidden.

18. Future Enhancements

The current EduGenie MVP focuses on the five core learning functions. Future versions could extend the system while keeping the existing modular architecture.

- Add optional user authentication and student profiles.
- Add persistent learning history using a suitable database.
- Add progress tracking and completion analytics.
- Support uploaded PDF or document learning material.
- Add multilingual learning support.
- Add richer quiz analytics and adaptive difficulty.
- Improve the local concept-explanation model and response quality.
- Add optional cloud deployment for remote access.
- Add automated API and integration tests.

These are future possibilities, not features claimed as part of the current MVP.

19. Conclusion

EduGenie is a modular learning assistant that combines Gemini-powered educational generation with a local concept-explanation model. The application brings Question Answering, Concept Explanation, Quiz Generation, Summarization, and Personalized Learning Path generation into a single web interface.

The project demonstrates a complete development workflow: model selection, secure API configuration, modular backend implementation, FastAPI integration, frontend development, local execution, functional testing, documentation, and GitHub version control.

The final MVP is intentionally focused. It does not use a persistent database, does not require user authentication, and is demonstrated locally through FastAPI and Uvicorn. These choices keep the current implementation straightforward while leaving clear opportunities for future enhancement.

Project status

Area	Status
Model Selection	Completed
Core Functionality	Completed
Frontend Development	Completed
Live Integration	Completed
Local Execution	Completed
Functional Testing	Completed
Documentation	Completed
GitHub Repository	Completed

EduGenie: Google Gemini Powered Learning Assistant

Team Lead: THAMIJUDEEN M

Team ID: 6ab39579211fa4fd5d28901b